

**MARS ROVER MANIPAL
RESEARCH AI TASKPHASE
Report 11**

Adarsh Inaganti
240905294
adarshinaganti006@gmail.com

1. Review questions

1.1 How is a dendrogram constructed?

A dendrogram is constructed during hierarchical clustering to visually represent how data points are grouped step-by-step based on similarity or distance.

1. Start with individual data points

Each observation begins as its own cluster.

2. Compute distance matrix

Calculate pairwise distances between all points using a distance metric.

- Euclidean distance
- Manhattan distance
- Cosine distance

This forms a distance matrix of shape (n, n) .

3. Choose a linkage method

A linkage criterion defines how distances between clusters are measured:

- **Single linkage:** Minimum distance between points
- **Complete linkage:** Maximum distance between points
- **Average linkage:** Mean distance between points
- **Ward's linkage:** Minimizes increase in within-cluster variance

4. Merge the closest clusters

- Identify the two clusters with the smallest distance
- Merge them into a new cluster
- Record the distance at which they merge

5. Update distances and repeat

- Recalculate distances between the new cluster and all remaining clusters
- Repeat merging until all points form one cluster

6. Draw the dendrogram

- **Leaves (bottom):** Individual data points
- **Branches:** Cluster merges
- **Height of merge:** Dissimilarity between clusters

Clusters can be extracted by drawing a horizontal cut across the dendrogram.

1.2 What are the different types of ReLU?

1. Standard ReLU

$$f(x) = \max(0, x)$$

- **Interpretation**
 - Outputs 0 for negative inputs
 - Linear for positive inputs
- **Pros**
 - Simple and fast
 - Reduces vanishing gradient problem
- **Cons**
 - Neurons can get stuck outputting 0 forever if weights push inputs negative (dying ReLU)

2. Leaky ReLU

$$f(x) = x \text{ if } x > 0$$
$$f(x) = \alpha x \text{ if } x \leq 0 \text{ where } \alpha \approx 0.01$$

- Allows a small negative slope instead of zero.
- **Pros**
 - Fixes dying ReLU
 - Keeps gradient alive for negative inputs
- **Cons**
 - α is fixed manually

3. Parametric ReLU (PReLU)

$$f(x) = x \text{ if } x > 0$$
$$f(x) = \alpha x \text{ if } x \leq 0 \text{ where } \alpha \text{ is learnable}$$

- The network learns how much negative slope it needs.
- **Pros**
 - More flexible than Leaky ReLU
 - Often better performance
- **Cons**
 - Due to extra parameters, there is a risk of overfitting

4. Randomized Leaky ReLU (RReLU)

$$\alpha \sim U(l, u)$$
$$f(x) = \alpha x \text{ for } x < 0$$

- Negative slope is random during training, fixed during testing.
- **Pros**
 - Regularization effect
 - Reduces overfitting
- **Cons**
 - Rarely used outside research

5. Exponential Linear Unit (ELU)

$$f(x) = x \text{ if } x > 0$$
$$f(x) = \alpha(e^x - 1) \text{ if } x \leq 0$$

- Smooth negative outputs instead of linear.
- **Pros**
 - Mean activations closer to zero
 - Faster convergence than ReLU
- **Cons**
 - Computationally expensive due to exponential function

6. Scaled Exponential Linear Unit (SELU)

$$f(x) = \lambda \cdot ELU(x)$$

- Automatically self-normalizes activations.
- **Pros**
 - Keeps mean ≈ 0 and variance ≈ 1
 - No batch normalization needed
- **Cons**
 - Requires special weight initialization
 - Works best in deep fully connected networks

7. ReLU6

$$f(x) = \min(\max(0, x), 6)$$

- Caps activation at 6.
- **Pros**
 - Used in mobile and low-precision models
 - Prevents exploding activations
- **Cons**
 - Limited output range

1.3 What is parametric ReLU?

- Parametric ReLU (PReLU) is a variant of the ReLU activation function.
- The slope for negative inputs is learned automatically during training instead of being fixed.

$$f(x) = x \text{ if } x > 0$$
$$f(x) = \alpha x \text{ if } x \leq 0 \text{ where } \alpha \text{ is learnable}$$

α is a trainable parameter.

- Unlike ReLU (which outputs 0 for all negative inputs) or
- Leaky ReLU (which uses a fixed small slope like 0.01)
- **PReLU lets the network decide how much information to pass for negative values.**

Advantages

- Prevents the dying ReLU problem
- Learns the optimal negative slope
- Often improves performance in deep CNNs

Disadvantages

- Introduces extra parameters
- Can cause overfitting on small datasets
- Slightly higher computational cost

Used in

- Deep convolutional neural networks
- When standard ReLU causes many neurons to die
- Large datasets where extra parameters are acceptable

2. Bias-Variance trade-off

2.1 Introduction

2.1.1 Definition

The bias-variance trade-off is a fundamental concept in machine learning that explains the source of errors in predictive models and highlights the inherent tension between model simplicity and model flexibility.

2.1.2 Total prediction error in ML

At a high level, the total prediction error of a machine learning model can be decomposed into three components:

$$\text{Total error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible error}$$

2.2 Bias

2.2.1 Definition

- Bias refers to the error introduced by simplifying assumptions made by the model to learn the target function.
- A model with high bias pays very little attention to the training data and oversimplifies the underlying relationships.

2.2.2 Characteristics of high bias

- Model is too simple
- Fails to capture important patterns in data
- Makes strong assumptions about data distribution
- Leads to underfitting

Examples

- Using a linear regression model for highly nonlinear data
- Shallow neural networks with very few hidden units
- Bias measures how far the average prediction of the model is from the true target value.
- High bias results in systematic error, meaning the model consistently makes incorrect predictions regardless of the training data size.

2.3 Variance

2.3.1 Definition

- Variance refers to the error caused by a model being overly sensitive to small fluctuations in the training dataset.
- A model with high variance learns noise and specific patterns in the training data that do not generalize to unseen data.

2.3.2 Characteristics of high variance

- Model is too complex
- Fits training data extremely well
- Highly sensitive to small changes in training data
- Leads to overfitting

Examples

- Deep neural networks trained on small datasets
- Decision trees without depth constraints
- Variance measures how much the model's predictions vary for different training datasets.
- High variance causes models to perform well on training data but poorly on validation or test data.

2.4 Trade-off between bias and variance

2.4.1 Relationship between bias and variance

Bias and variance have an inverse relationship.

- Reducing bias generally increases variance
- Reducing variance often increases bias

2.4.2 Trade-off

The inverse relationship creates a trade-off that must be carefully managed.

- Increasing model complexity (for example, adding more layers or parameters) reduces bias but increases variance.
- Conversely, simplifying the model reduces variance but increases bias.

The optimal model lies at the point where the sum of bias and variance is minimized, resulting in the lowest possible generalization error.

2.5 Diagnostic approach

The bias-variance trade-off can be observed by analyzing training and validation errors.

Model behaviour	Training error	Validation error	Interpretation
High bias	High	High	Underfitting
High variance	Low	High	Overfitting
Balanced	Low	Low	Good generalization

This diagnostic approach is widely used to identify whether a model requires more complexity, more data, or regularization.

2.6 Role of data size

Increasing the size of the training dataset helps in reducing variance, not bias.

- High variance models benefit significantly from more data.
- High bias models show little improvement even with more data.

Thus

- If the validation error decreases with more data, variance is the problem.
- If the error remains high, bias is the problem.

2.7 Managing the bias-variance trade-off

Several techniques are used to achieve a better balance.

2.7.1 Reducing bias

- Increase model complexity
- Add more features
- Use deeper neural networks

2.7.2 Reducing variance

- Increase training data
- Apply regularization (L1, L2, dropout)
- Use early stopping
- Simplify the model

The choice of strategy depends on whether the model is currently suffering from underfitting or overfitting.

2.8 Importance in deep learning

In deep learning, the bias–variance trade-off becomes even more critical due to

- Extremely large parameter spaces
- Sensitivity to hyperparameters
- Dependence on optimization techniques

Modern deep learning techniques aim to reduce bias and variance through

- Careful architecture design
- Regularization methods
- Advanced optimization algorithms.

3. Optimizers

3.1 Introduction

3.1.1 Definition

- Optimizers are a crucial component of training machine learning and deep learning models.
- They determine how the model's parameters (weights and biases) are updated during training in order to minimize the loss function.
- While the learning algorithm defines what is learned, the optimizer defines how efficiently learning occurs.

In deep neural networks, the choice of optimizer significantly affects

- Speed of convergence
- Stability of training
- Ability to escape poor local minima
- Final model performance

3.2 Role of optimizers in learning

- Training a neural network involves minimizing a loss (cost) function $L(w)$, which measures the discrepancy between predicted and actual outputs.
- Optimizers work by iteratively updating the model parameters in the direction that reduces this loss, using gradient information obtained via backpropagation.

3.2.1 Update rule

The generic update rule is

$$w_{i+1} = w_i - \alpha \nabla L(w_i)$$

- w_i are the parameters at iteration i
- α is the learning rate
- $\nabla L(w_i)$ is the gradient of the loss with respect to the parameters

The optimizer's main task is to control the learning rate, direction, and stability of these updates.

3.3 The foundation of optimizers (gradient descent)

- All modern optimizers are extensions or improvements over gradient descent.
- There are three common variants.

3.3.1 Batch Gradient Descent

- Uses the entire dataset to compute gradients
- Stable but computationally expensive
- Slow for large datasets

3.3.2 Stochastic Gradient Descent (SGD)

- Updates parameters using one data point at a time
- Faster updates but noisy
- High variance in parameter updates

3.3.3 Mini-batch Gradient Descent

- Uses small batches of data
- Balances stability and efficiency
- Most commonly used in practice

3.3.4 Limitations of gradient descent

- Slow convergence
- Oscillations in narrow valleys
- Sensitivity to learning rate choice
- Poor handling of non-convex loss surfaces

3.4 Challenges in optimization

Deep learning optimization is difficult due to

- High-dimensional parameter spaces
- Saddle points (flat regions with zero gradient)
- Ill-conditioned curvature (steep in one direction, flat in another)
- Noisy gradients in mini-batch training

Optimizers are designed to address these challenges by modifying how gradients are accumulated and applied.

3.5 Momentum-based optimization

Momentum introduces the idea of velocity to parameter updates. Instead of relying solely on the current gradient, it also considers past gradients to determine the update direction.

- Gradients are accumulated over time
- Updates gain speed in consistent directions
- Oscillations are dampened

Mathematically, momentum can be expressed as

$$\begin{aligned}v_i &= \beta v_{i-1} + (1 - \beta) \nabla L(w_i) \\w_{i+1} &= w_i - \alpha v_i\end{aligned}$$

- v_i is the velocity term
- β is the momentum coefficient (usually close to 0.9)

Advantages

- Faster convergence
- Reduced oscillations
- Better navigation of ravines in the loss surface

3.6 Adaptive learning rate methods

Adaptive optimizers adjust the learning rate individually for each parameter, instead of using a single global learning rate.

This is particularly useful when

- Features have different scales
- Some parameters learn faster than others
- Gradients vary significantly across dimensions

3.7 Adam (Adaptive Moment estimation)

Adam is one of the most widely used optimizers in deep learning because it combines the benefits of momentum and adaptive learning rates.

Adam maintains two running averages.

- First moment (mean of gradients)
- Second moment (uncentered variance of gradients)

These moments are bias-corrected and used to scale parameter updates.

3.7.1 Key properties

- Automatically adapts learning rate
- Works well with sparse gradients
- Requires little tuning
- Stable and fast convergence

Because of these advantages, Adam is often chosen as the default optimizer for deep neural networks.

3.8 Influence of optimizers on bias-variance trade-off

- Faster convergence can reduce underfitting
- Poor optimization can mimic high bias
- Excessively aggressive optimization may increase variance

Thus, optimization must be considered alongside

- Model architecture
- Regularization
- Dataset size

3.9 Importance in deep learning

In deep neural networks, optimizers are not just speed enhancers as they often determine whether training succeeds at all. A poor optimizer choice can lead to

- Vanishing or exploding gradients
- Extremely slow learning
- Failure to converge

Modern deep learning relies heavily on advanced optimizers to make training large-scale models feasible.